

Experiment 8: Heart Disease Prediction using Decision Tree, KNN, and Neural Network

Aim

To build and compare classification models using Decision Tree, K-Nearest Neighbors (KNN), and Neural Network algorithms for predicting heart disease, and evaluate their performance using accuracy, confusion matrix, and classification metrics.

Code

```
# HEART DISEASE PREDICTION USING DECISION TREE, KNN, AND NEURAL NETWORK
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load Heart Disease Dataset
df = pd.read_csv("heart.csv")
```

```
print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

```
# Assuming target column is 'target' (1 = disease, 0 = no disease)
print(df['target'].value_counts())
```

```
# Split features and target
X = df.drop('target', axis=1)
y = df['target']
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# ===== DECISION TREE =====
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
```

```
y_pred_dt = dt.predict(X_test)
```

```
print("=== Decision Tree ===")
```

```

print(f"Accuracy: {accuracy_score(y_test, y_pred_dt):.2f}")
print(classification_report(y_test, y_pred_dt))

sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap='Blues')
plt.title("Decision Tree - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

plt.figure(figsize=(20, 8))
plot_tree(dt, feature_names=X.columns, class_names=['No Disease', 'Disease'], filled=True)
plt.title("Decision Tree")
plt.show()

feat_imp = pd.Series(dt.feature_importances_, index=X.columns).sort_values(ascending=False)
feat_imp.plot(kind='bar')
plt.title("Feature Importance - Decision Tree")
plt.tight_layout()
plt.show()

# ===== KNN =====
from sklearn.neighbors import KNeighborsClassifier

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train_sc, y_train)

y_pred_knn = knn.predict(X_test_sc)

print("=== KNN ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f}")
print(classification_report(y_test, y_pred_knn))

sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, fmt='d', cmap='Greens')
plt.title("KNN - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# K vs Accuracy
k_range = range(1, 21)
k_scores = [KNeighborsClassifier(n_neighbors=k).fit(X_train_sc, y_train)
             .score(X_test_sc, y_test) for k in k_range]

plt.plot(k_range, k_scores, marker='o')
plt.xlabel("K value")
plt.ylabel("Accuracy")

```

```

plt.title("KNN Accuracy vs K")
plt.show()

# ===== NEURAL NETWORK =====
from sklearn.neural_network import MLPClassifier

nn = MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=500, random_state=42)
nn.fit(X_train_sc, y_train)

y_pred_nn = nn.predict(X_test_sc)

print("=== Neural Network ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_nn):.2f}")
print(classification_report(y_test, y_pred_nn))

sns.heatmap(confusion_matrix(y_test, y_pred_nn), annot=True, fmt='d', cmap='Oranges')
plt.title("Neural Network - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

plt.plot(nn.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Neural Network - Training Loss Curve")
plt.show()

# ===== MODEL COMPARISON =====
results = pd.DataFrame({
    'Model': ['Decision Tree', 'KNN', 'Neural Network'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_knn),
        accuracy_score(y_test, y_pred_nn)
    ]
})

results.set_index('Model')['Accuracy'].plot(kind='bar', ylim=(0.6, 1.0))
plt.title("Model Comparison - Accuracy")
plt.ylabel("Accuracy")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

print(results.to_string(index=False))

```

Output

(1025, 14)

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope \
0	52	1	0	125	212	0	1	168	0	1.0	2
1	53	1	0	140	203	1	0	155	1	3.1	0
2	70	1	0	145	174	0	1	125	1	2.6	0
3	61	1	0	148	203	0	1	161	0	0.0	2
4	62	0	0	138	294	1	1	106	0	1.9	1

	ca	thal	target
0	2	3	0
1	0	3	0
2	0	3	0
3	1	3	0
4	3	2	0

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1025 entries, 0 to 1024
```

```
Data columns (total 14 columns):
```

```
#   Column   Non-Null Count  Dtype
```

```
---  ---
```

```
0  age      1025 non-null  int64
1  sex      1025 non-null  int64
2  cp       1025 non-null  int64
3  trestbps 1025 non-null  int64
4  chol     1025 non-null  int64
5  fbs      1025 non-null  int64
6  restecg  1025 non-null  int64
7  thalach  1025 non-null  int64
8  exang    1025 non-null  int64
9  oldpeak  1025 non-null  float64
10 slope   1025 non-null  int64
11 ca      1025 non-null  int64
12 thal    1025 non-null  int64
13 target  1025 non-null  int64
```

dtypes: float64(1), int64(13)

memory usage: 112.2 KB

None

	age	sex	cp	trestbps	chol \
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000
std	9.072290	0.460373	1.029641	17.516718	51.59251
min	29.000000	0.000000	0.000000	94.000000	126.000000
25%	48.000000	0.000000	0.000000	120.000000	211.000000
50%	56.000000	1.000000	1.000000	130.000000	240.000000
75%	61.000000	1.000000	2.000000	140.000000	275.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000

	fbs	restecg	thalach	exang	oldpeak \
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	0.149268	0.529756	149.114146	0.336585	1.071512
std	0.356527	0.527878	23.005724	0.472772	1.175053
min	0.000000	0.000000	71.000000	0.000000	0.000000
25%	0.000000	0.000000	132.000000	0.000000	0.000000
50%	0.000000	1.000000	152.000000	0.000000	0.800000
75%	0.000000	1.000000	166.000000	1.000000	1.800000
max	1.000000	2.000000	202.000000	1.000000	6.200000

	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000
mean	1.385366	0.754146	2.323902	0.513171
std	0.617755	1.030798	0.620660	0.500070
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	2.000000	0.000000
50%	1.000000	0.000000	2.000000	1.000000
75%	2.000000	1.000000	3.000000	1.000000
max	2.000000	4.000000	3.000000	1.000000

target

1 526

0 499

Name: count, dtype: int64

=== Decision Tree ===

Accuracy: 0.80

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

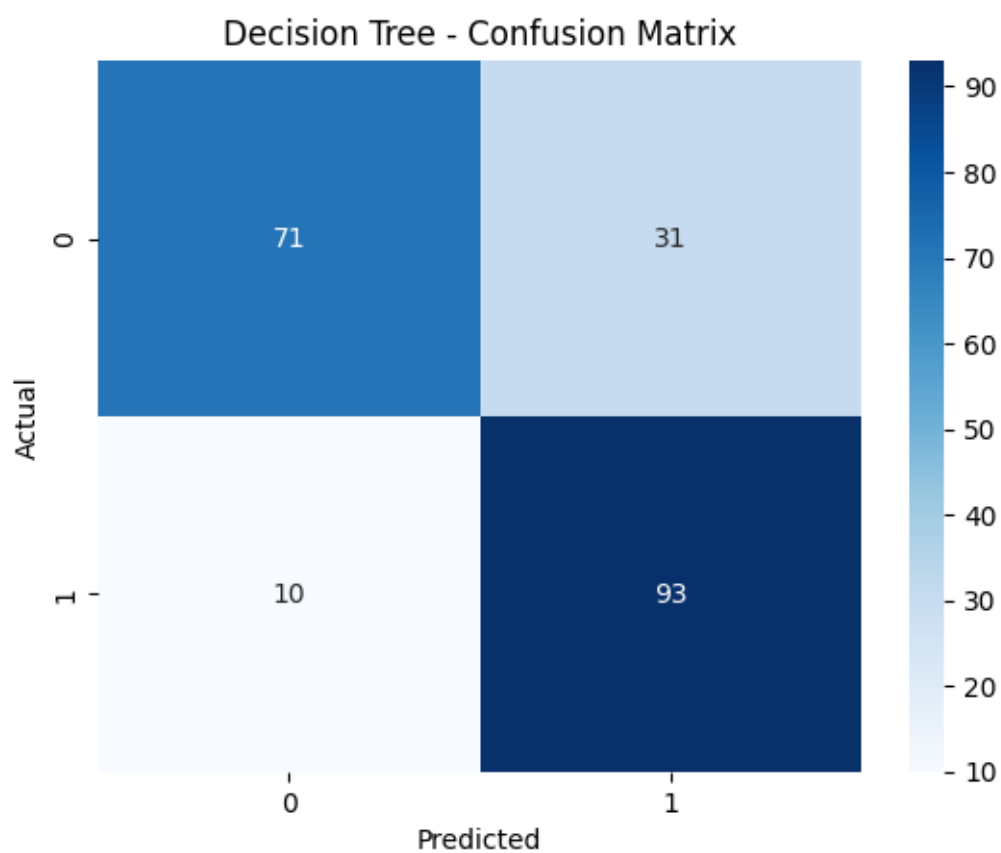
0	0.88	0.70	0.78	102
---	------	------	------	-----

1	0.75	0.90	0.82	103
---	------	------	------	-----

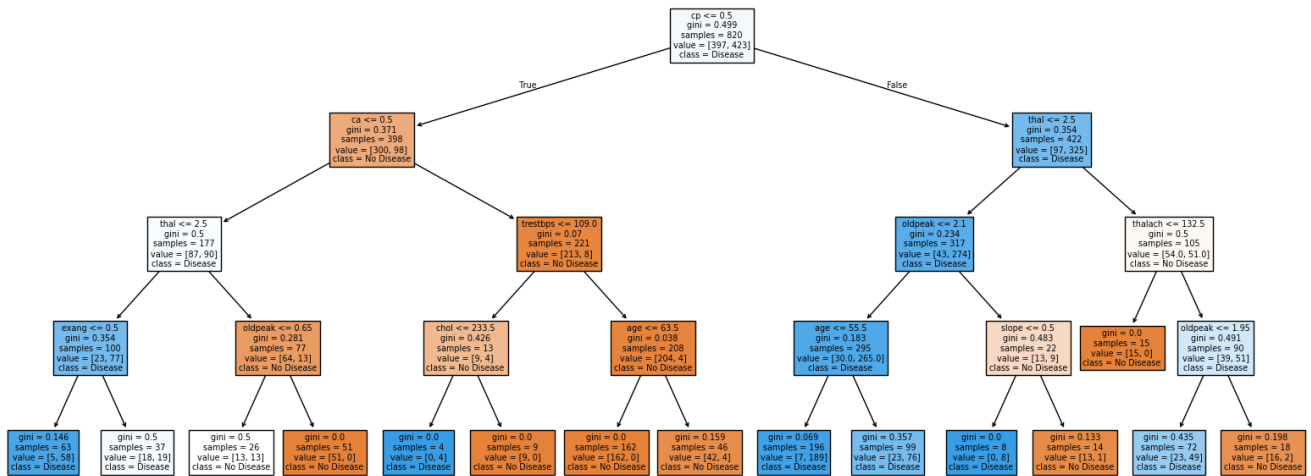
accuracy		0.80	205
----------	--	------	-----

macro avg	0.81	0.80	0.80	205
-----------	------	------	------	-----

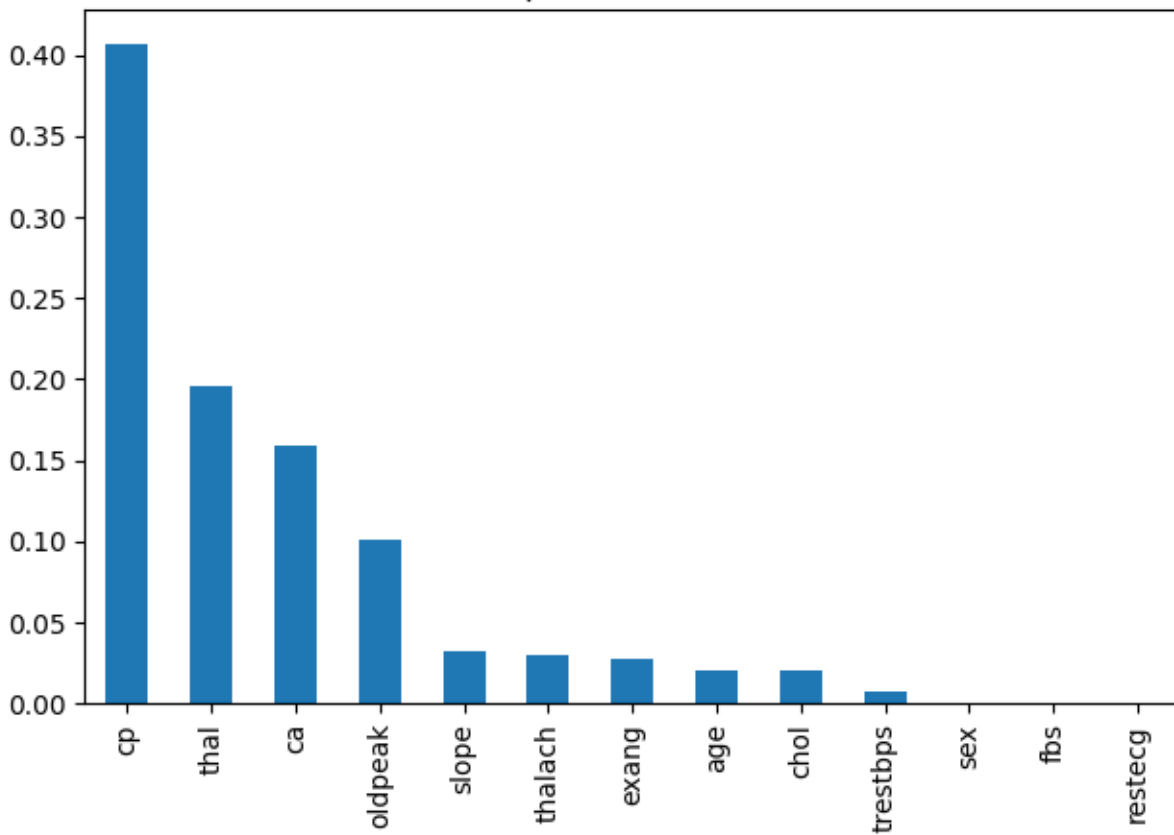
weighted avg	0.81	0.80	0.80	205
--------------	------	------	------	-----



Decision Tree



Feature Importance - Decision Tree



=== KNN ===

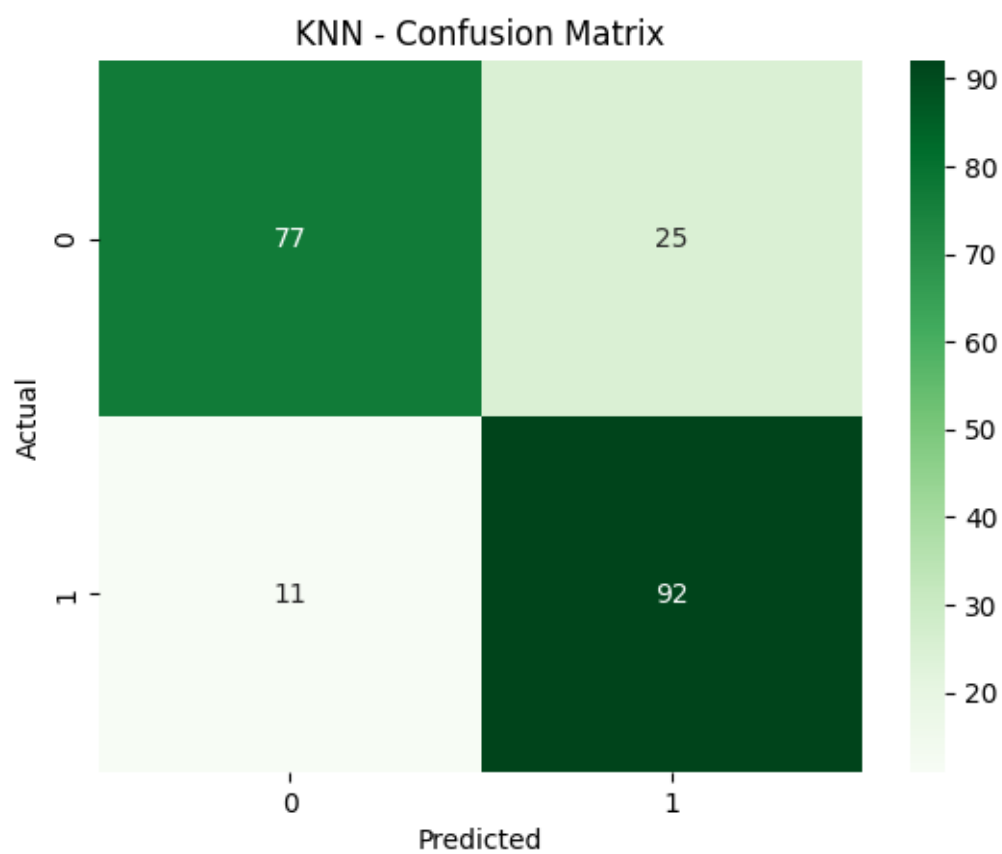
Accuracy: 0.82

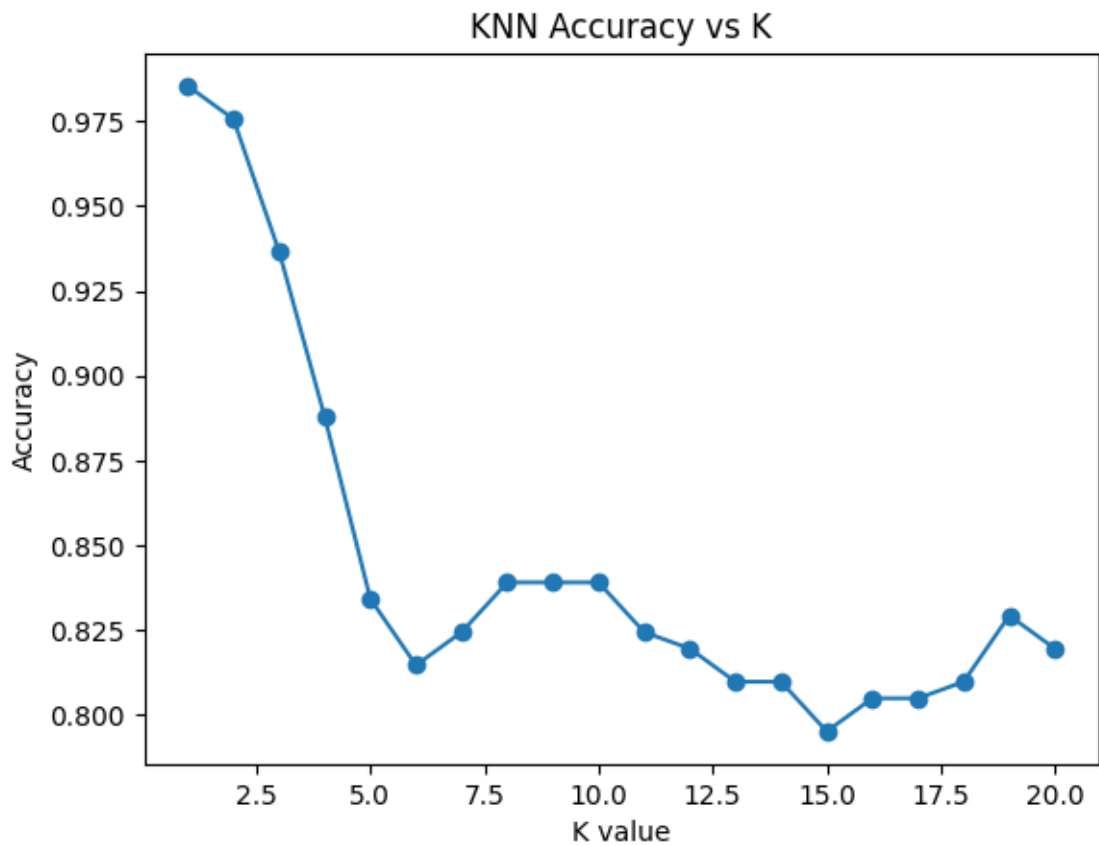
precision recall f1-score support

0 0.88 0.75 0.81 102

1 0.79 0.89 0.84 103

accuracy		0.82	205	
macro avg	0.83	0.82	0.82	205
weighted avg	0.83	0.82	0.82	205





=== Neural Network ===

Accuracy: 0.98

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.96	1.00	0.98	102
---	------	------	------	-----

1	1.00	0.96	0.98	103
---	------	------	------	-----

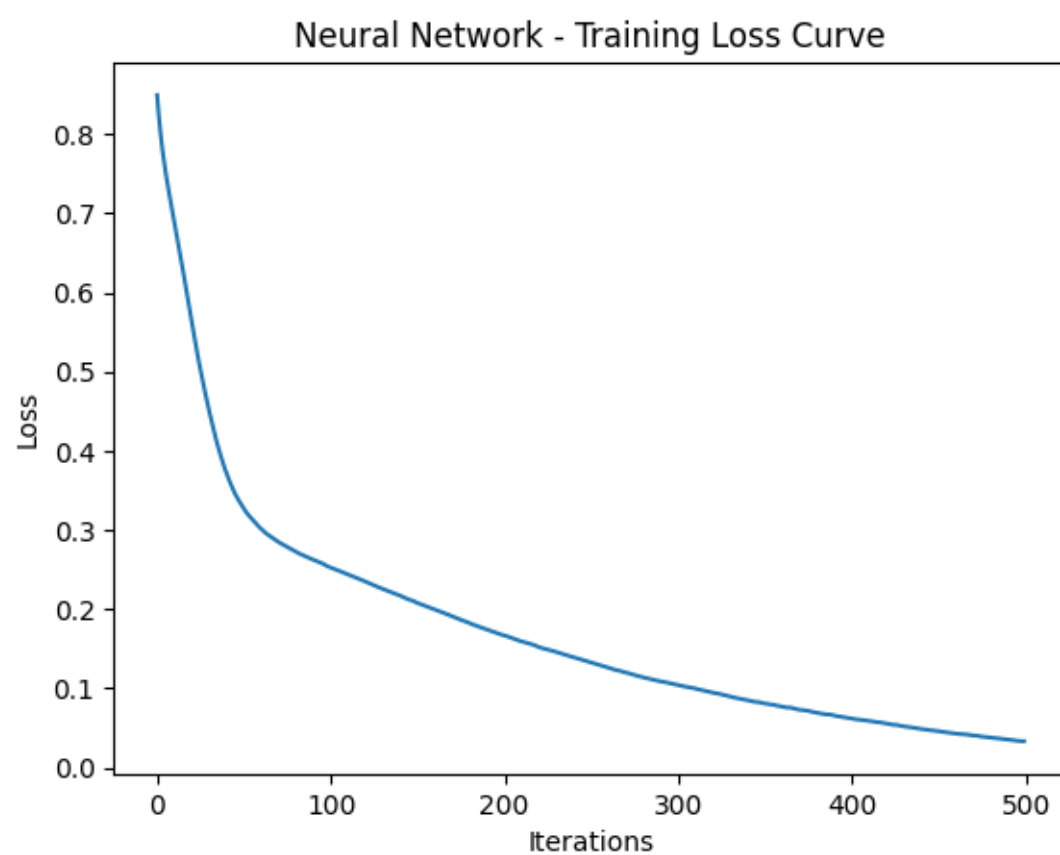
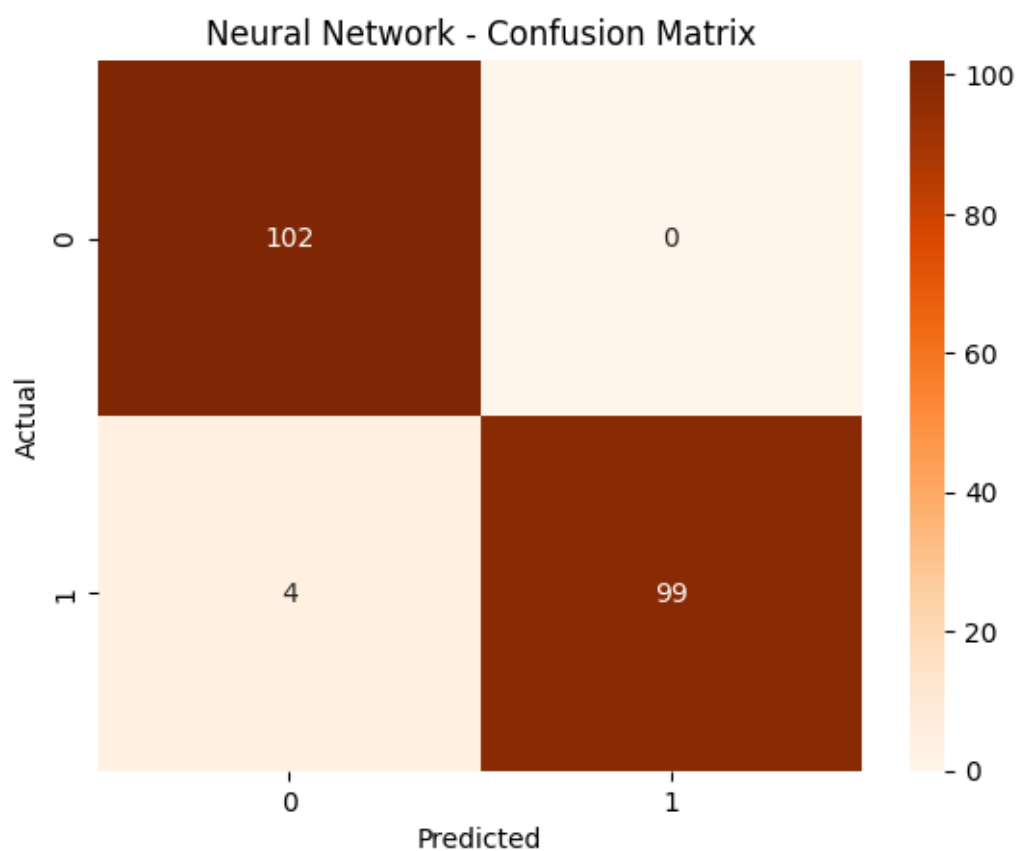
accuracy		0.98	205
----------	--	------	-----

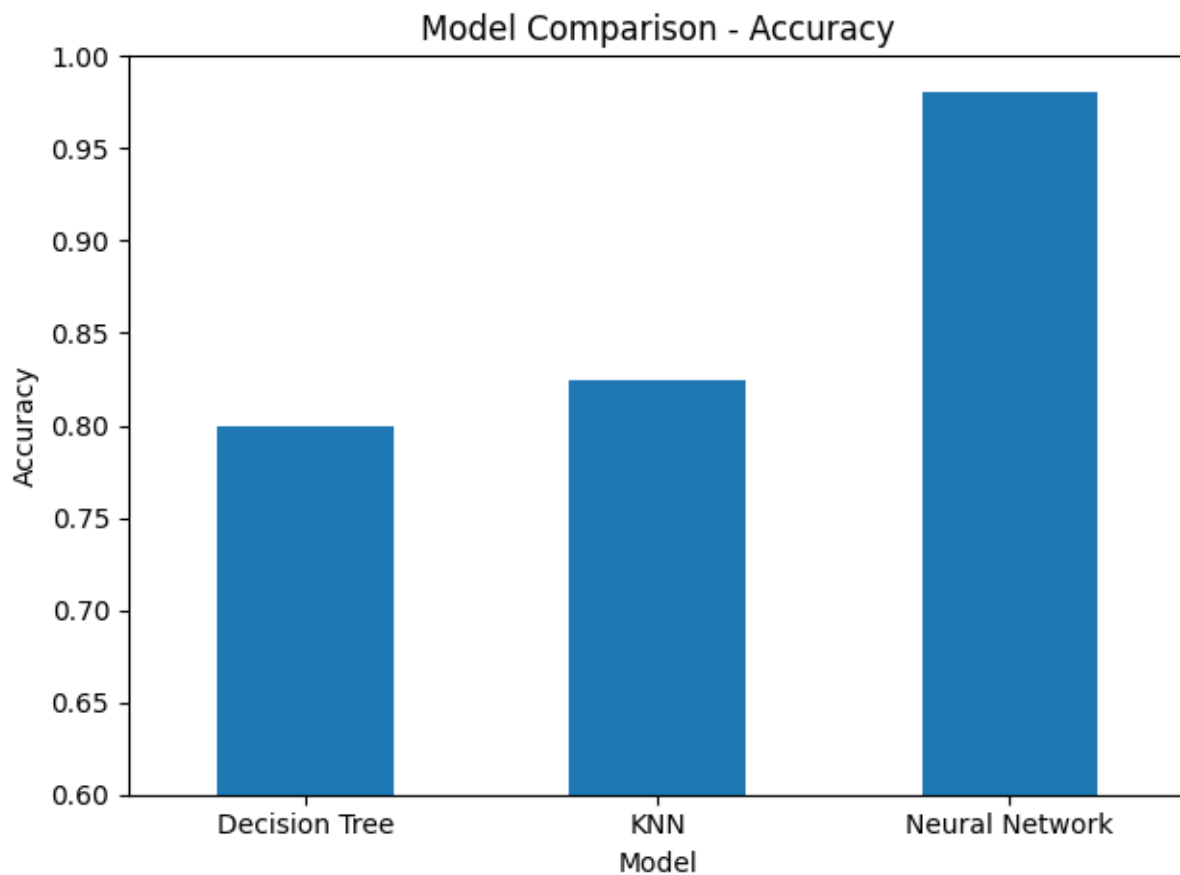
macro avg	0.98	0.98	0.98	205
-----------	------	------	------	-----

weighted avg	0.98	0.98	0.98	205
--------------	------	------	------	-----

/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691:
 ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and the optimization hasn't
 converged yet.

warnings.warn(





Model Accuracy

Decision Tree 0.800000

KNN 0.824390

Neural Network 0.980488

Result

The dataset was used to build and evaluate three classification models: Decision Tree, KNN, and Neural Network. Each model was trained and tested, and their performance was compared using accuracy, confusion matrices, and classification reports. Feature importance and visualization techniques provided insights into the data. Among the models, performance varied, and the comparison helped identify the most effective algorithm for heart disease prediction.